

being explicit. Explicitly capturing naming your slugs as ‘slug’ can be advantageous though when dealing with class-based views (which we will cover later in this text). As such we will leave pattern names except when capturing slugs.

Our URL Configuration

We’ve discussed the URL structure we want to use in the previous chapter, so now let’s get on to implementing it!

Let’s look at all the pieces of content and see how their URLs will be implemented.

For articles we need to have a main ‘articles/’ URL to list all articles, and a ‘articles/<article-slug>’ URL for individual articles:

- `^articles/$`
- `^articles/(?P<slug>[\w-]+$)`

We can also make it possible to browse articles by date, we’ve already seen these before, but here they are again:

- `^archive/([0-9]{4})/$`
- `^archive/([0-9]{4})/([0-9]{2})/$`
- `^archive/([0-9]{4})/([0-9]{2})/([0-9]{2})/$`

Like for articles, we need to have a ‘categories/’ page to list all categories and a ‘categories/<category-name>’ page for each category. We need similar pages for tags, and galleries.

- `^categories/$`
- `^categories/(?P<slug>[\w-]+$)`
- `^tags/$`
- `^tags/(?P<slug>[\w-]+$)`
- `^galleries/$`
- `^galleries/(?P<slug>[\w-]+$)`

Images don’t have a listing page, but they still need a page for each image.

- `^image/(?P<slug>[\w-]+$)`

Finally, we have Authors. Now Authors don’t have a slug! However each author does have a unique username and serial ID. We can use one of these. We will use the username since the user’s serial ID is an internal piece of data that the user shouldn’t need to be aware of, and should preferably be kept hidden. How we map the slug here to a username is something you will learn in the next chapter.

- `^authors/$`
- `^authors/(?P<slug>[\w-]+$)`